🧪 Leakage Stress-Test Checklist

Run this before modeling, evaluation, or deployment
Goal: Ensure the system only uses information that would be available at the moment a real decision is made.

1️⃣ Fix the Decision Moment (Do This First)

☐ Can you state the **exact moment** the decision is made?(e.g., "at the start of the billing cycle," "when a ticket is opened," "daily at 9am")
☐ Is this moment **earlier than** the outcome you're predicting?
☐ Does *every* feature clearly exist **at or before** this moment?
🔴 **Red flag:** "We'll figure timing out later."

2️⃣ Observation vs Outcome Windows (Hard Boundary)

☐ Is there a clearly defined **observation window**?(What the model is allowed to see)
☐ Is there a clearly defined **outcome window**?(What happens after the decision)
☐ Are these windows **strictly separated**?
🔴 **Red flag:** Features computed over "the full dataset" or "the next X days."

3️⃣ Feature-Level Leakage Checks (Run for Every Feature)

For **each feature**, ask:
☐ Could this value exist **before** the decision is made?
☐ Does this feature depend on:

- human review?
- escalation?
- closure?
- intervention?

☐ Would this feature change **because** the decision was acted on?
🔴 **Red flag:** "It's available eventually."

4️⃣ Label & Proxy Leakage Detection

☐ Does any feature:

- directly encode the label?

- summarize the outcome?
- depend on post-outcome state?

☐ Is this feature:

- assigned after the event?
- derived from business processes triggered by the event?

☐ Would a human need to *already know the outcome* to generate it?

🔴 **Red flag:** Feature names like `final_status`, `resolution_code`, `closure_reason`.

5️⃣ Aggregation & Normalization Checks

☐ Are group-level statistics computed using:

- only past data?
- or the full dataset?

☐ Are normalization parameters (means, std devs):

- fit only on historical data?
- frozen at decision time?

☐ Are cohort-level features updated **after** outcomes occur?

🔴 **Red flag:** Global statistics recomputed across train + test.

6️⃣ Evaluation & Splitting Sanity Check

☐ Is data split **temporally**, not randomly?

☐ Does training data come strictly **before** validation data?

☐ Are entities (customers, accounts) prevented from leaking across splits?

🔴 **Red flag:** Random splits for time-dependent decisions.

7️⃣ Feedback Loop & Intervention Awareness (Advanced)

☐ Does the system's output influence future inputs?

☐ Are post-intervention behaviors treated as new evidence?

☐ Is there a mechanism to distinguish:

- observed behavior
- behavior caused by the system itself?

🔴 **Red flag:** "The model will just retrain periodically."

8️⃣ "Too Good to Be True" Test

☐ Does performance feel implausibly high?

☐ Does a simple model outperform expectations dramatically?

☐ Does performance collapse in live or holdout data?

🔴 **Red flag:** Early success without skepticism.

9️⃣ The Single Most Important Question (Always Ask)

For **any feature**, ask:

"Could I have known this at the exact moment the decision was made?"

If:

- ❌ No → leakage
- 🤷 Unsure → high risk
- ✅ Yes → proceed (but still document assumptions)

1️⃣0️⃣ Document Assumptions (This Saves You Later)

☐ Have you written down:

- the decision timestamp
- allowed data sources
- forbidden features
- known leakage risks?

☐ Could someone new to the project detect leakage from your documentation?

🔴 **Red flag:** "It's obvious from the code."

🧠 How to Use This in Practice

- Run this **before feature engineering is finalized**
- Run it **again before evaluation**
- Run it **again before deployment**
- Use it as a *thinking tool*, not a formality

Even catching **one** leakage risk early can save months of rework.

One-Sentence Anchor (Memorize This)

Leakage is not about bad code — it's about allowing the model to see a version of reality that never exists at decision time.

You're now equipped to spot one of the most common reasons DS systems fail *after* looking successful.